COMPREHENSIVE DEVELOPER CODEX

# README_ULTIMATE.md

## COMPLETE STEP-BY-STEP IMPLEMENTATION MANUAL FOR PRATHAM-CHIKITSE

### 🎯 Purpose & Scope

This document functions as an absolute build codex. It contains the exact, uncompromised step-by-step instructions, logical workflows, and code structures required to build all screens, persistent features, and components of the **Pratham-Chikitse** Android app in Kotlin from scratch.

## Phase 1: Project Setup & Initialization

1. Launch **Android Studio** and select **New Project** -> **Empty Views Activity**.
2. Configure: Package: `org.vtu.mindmatrix.prathamchikitse`, Language: `Kotlin`, Minimum SDK: `API 26`.
3. Enable ViewBinding in `app/build.gradle.kts` within the `android { ... }` scope:

```
buildFeatures { viewBinding = true }
```

4. Sync Gradle files immediately.

## Phase 2: Defining Data Models & Local Storage

To establish type-safe architecture, create three data schemas within `data/model/`:

### 1. EmergencyStep.kt

Holds the specific details for individual procedure screens inside the ViewPager2 carousel.

```
package org.vtu.mindmatrix.prathamchikitse.data.model

data class EmergencyStep(
    val stepNumber: Int,
    val titleEn: String,
    val titleKn: String,
    val instructionEn: String,
    val instructionKn: String,
    val isDo: Boolean, // green DO badge or red DONT badge
    val illustrationResName: String // drawable reference key
)
```

### 2. EmergencyCase.kt

Groups a collection of step processes under an explicit parent category.

```
package org.vtu.mindmatrix.prathamchikitse.data.model

data class EmergencyCase(
    val id: Int,
    val nameEn: String,
    val nameKn: String,
    val iconResName: String,
    val severity: String,
    val steps: List<EmergencyStep>
)
```

### 3. Hospital.kt

Stores pre-configured clinical endpoints for offline mapping and click-to-dial functionality.

```
package org.vtu.mindmatrix.prathamchikitse.data.model

data class Hospital(
    val id: Int,
    val nameEn: String,
    val nameKn: String,
    val distanceKm: Double,
    val phone: String,
    val addressEn: String,
    val addressKn: String
)
```

## Phase 3: Screen 1 — Security & Auth Flow (AuthActivity.kt)

Restricts access via a clean, mock verification process. Saves the authentication state locally for rapid landing.

### Step-by-Step UI Layout Creation (activity_auth.xml)

Add a clean vertical layout containing a phone number edit text, a primary button, a verification pin input, and a final access verification button:

```
<!-- Code structure matches Section 10 in standard SOP -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp"
    android:background="#f8f9fa">
    <!-- Input views config and Verification logic goes here -->
</LinearLayout>
```

### Logical Workflow (AuthActivity.kt)

- Instantiate your UI ViewBinding engine.
- When `btnGetOtp` is clicked, validate phone syntax length (>= 10 digits). If valid, set `etOtp` and `btnVerify` visibility to `View.VISIBLE`.
- Upon clicking `btnVerify`, match the input pattern. If validation succeeds, save the active preference state to `EncryptedSharedPreferences` and intent-route to `MainActivity`.

## Phase 4: Screen 2 — Main Interactive Dashboard (MainActivity.kt)

Visual entry point displaying localized emergency options within an accessible grid framework.

### Step-by-Step UI Layout (activity_main.xml)

Design a top bar containing the app title and language switcher, a dynamic `RecyclerView` configured as a 2xN grid, and a bottom persistent button for offline clinical locations.

### Logical Workflow (MainActivity.kt)

- Instantiate `EmergencyViewModel` to fetch cases from your local offline repository.
- Observe LiveData flows. Whenever changes are detected or the language state updates, submit the matching string sets to your custom adapter class.
- Add a button toggle click-listener that updates the active locale settings dynamically between `"en"` and `"kn"`.

## Phase 5: Screen 3 — Step-by-Step Guide View (DetailActivity.kt)

Crucial emergency component built to output structured procedures seamlessly via slide views.

### Step-by-Step UI Layout (activity_detail.xml)

Integrate a lightweight top header containing a back navigation arrow, a prominent full-width `ViewPager2` component to load procedural steps, and a bottom bar housing the Text-To-Speech (TTS) trigger button.

### Logical Workflow & TTS Management (DetailActivity.kt)

- Bind your step layouts to the `ViewPager2` using a custom adapter.
- In the `onPageSelected` callback of your ViewPager2, call `ttsHelper.stop()` to prevent old audio instructions from overlapping.

- When the speaker button is clicked, pass the current step's text instructions (localized based on the active language) to your helper wrapper to read aloud.

## Phase 6: Compilation, Verification & Quality Control

### Local Compilation Steps

1. Verify all XML resources and vector icons exist inside `res/drawable/`.
2. Open the terminal window in Android Studio and compile the debug APK using:

```
./gradlew assembleDebug
```

3. Fix any compilation errors immediately by verifying ViewBinding paths and imports.

> **🚀 Final Compliance Metric Check**
>
> Your finalized APK must load in under **500ms**, function entirely offline without calling network requests, and support smooth, bilingual Kannada/English instructions natively.